

DOM y renderizado dinámico en JavaScript

Enfoque de la sesión

En esta sesión vas a trabajar el paso que conecta la lógica de JavaScript con la interfaz visible: **leer el documento, seleccionar elementos, crear nuevos nodos y renderizar información de forma dinámica**. El objetivo no es manipular el DOM de forma mecánica, sino comprender cómo una estructura de datos puede convertirse en una interfaz y cómo esa interfaz puede actualizarse con criterio, claridad y cierto cuidado por el rendimiento y la seguridad.

1. Introducción

Hasta ahora has trabajado con variables, funciones, arrays, objetos y transformación de datos. Eso te ha permitido **representar** y **procesar** información, pero todavía falta una pieza esencial para construir aplicaciones en el navegador: **mostrar esa información en la página** y hacerlo de manera dinámica.

Cuando una aplicación web enseña una lista de productos, pinta una galería, muestra una tarjeta con información, actualiza un mensaje o cambia una sección completa de la interfaz, está trabajando con el **DOM**. El DOM es el puente entre el código JavaScript y la estructura HTML que el navegador ha interpretado y convertido en una representación manipulable.

Comprender esta relación es fundamental porque te permite:

- localizar elementos existentes;
- leer o modificar su contenido;
- añadir o eliminar nodos;
- construir interfaz a partir de datos;
- entender mejor qué problema resuelve después React.

En esta sesión no se trata solo de aprender métodos como `querySelector()` o `append()`. Se trata de asumir una idea clave: **la interfaz puede generarse desde los datos**. Esa idea será central en todo lo que venga después.

2. Qué es el DOM

2.1. Definición general

DOM son las siglas de *Document Object Model*. Es la representación estructurada del documento HTML como un árbol de nodos que JavaScript puede leer y modificar.

Cuando el navegador carga un HTML, no trabaja con él solo como texto plano. Lo interpreta y construye una estructura interna sobre la que luego JavaScript puede actuar.

2.2. Idea de árbol

El DOM puede entenderse como una jerarquía:

- un documento raíz;
- elementos dentro del documento;
- elementos anidados dentro de otros;
- relaciones de padre, hijo y hermano.

Por ejemplo, en un HTML como este:

```
<body>
  <main>
    <h1>Título</h1>
    <p>Texto</p>
  </main>
</body>
```

La relación estructural sería:

- `body` contiene a `main`;
- `main` contiene a `h1` y `p`;
- `h1` y `p` son hijos del mismo padre.

2.3. Por qué importa esta idea

Comprender el DOM como árbol ayuda a:

- seleccionar con más criterio;
- entender la posición de cada elemento;
- modificar interfaz sin perder contexto;
- trabajar después con eventos y delegación.

El DOM no es solo “lo que se ve en pantalla”.
Es la **estructura programable** del documento HTML.

Referencias específicas del subtema

- MDN Web Docs — Introduction to the DOM
 - MDN Web Docs — Document Object Model (DOM)
-

3. `document` como punto de entrada

3.1. Qué representa `document`

En el navegador, `document` es el objeto que representa el documento HTML actual y actúa como punto de entrada al DOM.

```
console.log(document);
```

A partir de `document`, puedes buscar elementos, crear nodos y modificar partes de la página.

3.2. Acceso a información general

```
console.log(document.title);  
console.log(document.body);
```

3.3. Idea práctica

La mayoría de operaciones iniciales sobre el DOM comienzan desde `document` o desde un elemento previamente seleccionado.

Referencias específicas del subtema

- MDN Web Docs — `document`
-

4. Selección de elementos

4.1. `getElementById()`

Permite seleccionar un elemento por su identificador.

```
<h1 id="titulo-principal">Catálogo</h1>
```

```
const titulo = document.getElementById("titulo-principal");  
console.log(titulo);
```

Es útil cuando el elemento tiene un `id` único y claro.

4.2. `querySelector()`

Permite seleccionar el primer elemento que coincida con un selector CSS.

```
<h1 class="titulo">Catálogo</h1>
```

```
const titulo = document.querySelector(".titulo");  
console.log(titulo);
```

También puedes seleccionar por etiqueta, id, clase o selectores más complejos.

```
const primerParrafo = document.querySelector("p");  
const boton = document.querySelector("#boton-enviar");  
const tarjeta = document.querySelector(".tarjeta.destacada");
```

4.3. `querySelectorAll()`

Permite seleccionar todos los elementos que coincidan con un selector CSS.

```
<ul>  
  <li class="producto">Teclado</li>  
  <li class="producto">Ratón</li>  
  <li class="producto">Monitor</li>  
</ul>
```

```
const productos = document.querySelectorAll(".producto");  
console.log(productos);
```

Devuelve una colección de nodos sobre la que puedes iterar.

```
const productos = document.querySelectorAll(".producto");

productos.forEach((producto) => {
  console.log(producto.textContent);
});
```

4.4. closest()

Permite buscar el ancestro más cercano que cumpla un selector.

```
<article class="tarjeta">
  <button class="eliminar">Eliminar</button>
</article>
```

```
const boton = document.querySelector(".eliminar");
const tarjeta = boton.closest(".tarjeta");

console.log(tarjeta);
```

Esto resulta especialmente útil cuando más adelante trabajes con eventos en estructuras anidadas.

Criterio de uso

En JavaScript moderno, `querySelector()` y `querySelectorAll()` suelen ser las herramientas más versátiles para seleccionar elementos, porque permiten aprovechar la lógica de los selectores CSS que ya conoces.

Referencias específicas del subtema

- MDN Web Docs — `Document.getElementById()`
- MDN Web Docs — `Document.querySelector()`
- MDN Web Docs — `Document.querySelectorAll()`
- MDN Web Docs — `Element.closest()`

5. Leer y modificar contenido

5.1. `textContent`

`textContent` permite leer o cambiar el contenido textual de un nodo.

```
<p id="mensaje">Texto original</p>
```

```
const mensaje = document.getElementById("mensaje");

console.log(mensaje.textContent);
mensaje.textContent = "Texto actualizado";
```

Es una opción segura y muy útil cuando solo necesitas trabajar con texto.

5.2. `innerHTML`

`innerHTML` permite leer o sustituir el contenido HTML interno de un elemento.

```
<div id="contenedor"></div>
```

```
const contenedor = document.getElementById("contenedor");

contenedor.innerHTML = "<p>Contenido insertado</p>";
```

5.3. Diferencia básica entre ambos

Propiedad	Uso principal	Observación
<code>textContent</code>	texto plano	más segura para texto
<code>innerHTML</code>	insertar o leer HTML interno	más potente, pero con más riesgos

5.4. Cuándo preferir `textContent`

Si lo que necesitas es mostrar texto proveniente de datos, `textContent` suele ser la opción preferible.

```
const nombreUsuario = "Lucía";
const salida = document.querySelector("#salida");

salida.textContent = nombreUsuario;
```

Referencias específicas del subtema

- MDN Web Docs — `Node.textContent`
 - MDN Web Docs — `Element.innerHTML`
-

6. Modificar clases y atributos

6.1. `classList`

`classList` permite trabajar con las clases CSS de un elemento.

```
<div id="tarjeta" class="producto"></div>
```

```
const tarjeta = document.getElementById("tarjeta");

tarjeta.classList.add("destacada");
tarjeta.classList.remove("producto");
tarjeta.classList.toggle("visible");
```

6.2. Ventajas de `classList`

- evita manipular manualmente la cadena de clases;
- hace más claro el cambio visual;
- conecta bien JavaScript con el CSS.

6.3. Atributos

Puedes leer o modificar atributos con distintos mecanismos.

```

```

```
const avatar = document.getElementById("avatar");

console.log(avatar.getAttribute("src"));
avatar.setAttribute("alt", "Imagen de perfil actualizada");
```

6.4. Propiedades y atributos

A menudo un elemento HTML tiene atributos en el marcado, pero en JavaScript trabajas también con propiedades del objeto DOM.

```
const avatar = document.getElementById("avatar");

console.log(avatar.src);
console.log(avatar.alt);
```

6.5. Diferencia conceptual

- el **atributo** pertenece al HTML;
- la **propiedad** pertenece al objeto DOM ya interpretado por el navegador.

En muchos casos ambos se relacionan, pero no siempre son idénticos en comportamiento.

Precisión importante

En el trabajo cotidiano no siempre tendrás que profundizar en cada matiz entre atributo y propiedad, pero sí conviene entender que no son exactamente lo mismo y que el navegador puede normalizar o transformar ciertos valores al representarlos como propiedades.

Referencias específicas del subtema

- MDN Web Docs — `Element.classList`
- MDN Web Docs — `Element.setAttribute()`
- MDN Web Docs — `Element.getAttribute()`

7. Crear, insertar y eliminar elementos

7.1. `createElement()`

Permite crear un nuevo elemento en memoria.

```
const tarjeta = document.createElement("article");
console.log(tarjeta);
```

Ese elemento existe, pero todavía no aparece en la página hasta que lo insertas.

7.2. Configurar el nodo creado


```
const tarjeta = document.createElement("article");
tarjeta.classList.add("tarjeta");
tarjeta.textContent = "Nuevo producto";
```

7.3. append()

Inserta uno o varios nodos al final del elemento contenedor.

```
<section id="lista-productos"></section>
```

```
const lista = document.getElementById("lista-productos");
const tarjeta = document.createElement("article");

tarjeta.textContent = "Teclado";
lista.append(tarjeta);
```

7.4. prepend()

Inserta uno o varios nodos al principio del contenedor.

```
const lista = document.getElementById("lista-productos");
const mensaje = document.createElement("p");

mensaje.textContent = "Productos destacados";
lista.prepend(mensaje);
```

7.5. remove()

Elimina un nodo del DOM.

```
const mensaje = document.querySelector(".mensaje-error");
mensaje.remove();
```

Idea clave

Una interfaz dinámica se basa en estas operaciones: **crear**, **insertar**, **actualizar** y **eliminar** nodos según el estado de los datos.

Referencias específicas del subtema

- MDN Web Docs — `Document.createElement()`
 - MDN Web Docs — `Element.append()`
 - MDN Web Docs — `Element.prepend()`
 - MDN Web Docs — `Element.remove()`
-

8. Renderizado dinámico a partir de datos

8.1. Qué significa renderizar

Renderizar consiste en convertir información en una representación visible dentro de la interfaz.

Si tienes datos como estos:

```
const productos = [  
  { nombre: "Teclado", precio: 30 },  
  { nombre: "Ratón", precio: 15 },  
  { nombre: "Monitor", precio: 200 }  
];
```

Puedes usarlos para crear elementos visuales en el DOM.

8.2. Ejemplo básico de renderizado dinámico

```
<section id="lista-productos"></section>
```

```
const productos = [  
  { nombre: "Teclado", precio: 30 },  
  { nombre: "Ratón", precio: 15 },  
  { nombre: "Monitor", precio: 200 }  
];  
  
const lista = document.getElementById("lista-productos");  
  
productos.forEach((producto) => {  
  const tarjeta = document.createElement("article");  
  const titulo = document.createElement("h2");  
  const precio = document.createElement("p");  
  
  titulo.textContent = producto.nombre;  
  precio.textContent = `${producto.precio} €`;
```

```
tarjeta.append(titulo, precio);  
lista.append(tarjeta);  
});
```

8.3. Qué se está haciendo realmente

- leer una colección de datos;
- recorrerla;
- crear nodos para cada elemento;
- insertar esos nodos en el contenedor.

8.4. Relación con sesiones anteriores

Aquí se conectan varias piezas ya trabajadas:

- arrays;
- objetos;
- `forEach()` ;
- transformación de datos;
- template literals.

Criterio pedagógico

Cuando un listado se genera desde datos, la interfaz deja de depender de HTML copiado manualmente y pasa a depender de la estructura de la información. Esa es una idea central de la programación de interfaces moderna.

Referencias específicas del subtema

- MDN Web Docs — `Document.createElement()`
- MDN Web Docs — `Element.append()`

9. Vaciar y volver a pintar contenido

9.1. Por qué hace falta

En muchas interfaces, antes de volver a renderizar contenido necesitas limpiar el estado visual anterior.

9.2. Ejemplo simple

```
const lista = document.getElementById("lista-productos");

lista.innerHTML = "";
```

Después puedes volver a insertar contenido actualizado.

9.3. Cuándo resulta útil

- al aplicar filtros;
- al cambiar una búsqueda;
- al recargar una lista;
- al rehacer una parte completa de la interfaz.

9.4. Precaución de diseño

Vaciar y reconstruir un bloque completo puede ser una estrategia válida en ejemplos y miniaplicaciones, pero conviene entender que no siempre es la solución más eficiente en interfaces complejas. Aun así, como punto de partida pedagógico resulta útil para comprender el ciclo de renderizado.

Referencias específicas del subtema

- MDN Web Docs — `Element.innerHTML`

10. DocumentFragment e inserción más eficiente

10.1. Qué problema resuelve

Si insertas muchos nodos uno a uno en el DOM, cada inserción puede implicar trabajo de actualización interna del documento. `DocumentFragment` permite construir un conjunto de nodos fuera del árbol principal y añadirlo después de una vez.

10.2. Ejemplo básico

```
const productos = [
  { nombre: "Teclado", precio: 30 },
  { nombre: "Ratón", precio: 15 },
  { nombre: "Monitor", precio: 200 }
];
```

```
const lista = document.getElementById("lista-productos");
const fragmento = document.createDocumentFragment();

productos.forEach((producto) => {
  const tarjeta = document.createElement("article");
  const titulo = document.createElement("h2");
  const precio = document.createElement("p");

  titulo.textContent = producto.nombre;
  precio.textContent = `${producto.precio} €`;

  tarjeta.append(titulo, precio);
  fragmento.append(tarjeta);
});

lista.append(fragmento);
```

10.3. Cuándo merece la pena mencionarlo

En esta sesión no necesitas convertir `DocumentFragment` en una obsesión técnica, pero sí conviene conocerlo como una herramienta útil cuando construyes varias inserciones de una sola vez.

Nivel esperado

Lo importante no es memorizar todos los matices de rendimiento, sino entender que existe una diferencia entre modificar el DOM repetidamente y preparar una inserción agrupada.

Referencias específicas del subtema

- MDN Web Docs — `Document.createDocumentFragment()`
- MDN Web Docs — `DocumentFragment`

11. `innerHTML` frente a `createElement()` : claridad, control y seguridad

11.1. La comodidad de `innerHTML`

`innerHTML` resulta cómodo porque permite insertar estructuras completas con una sola asignación.

```
const tarjeta = document.querySelector("#tarjeta");

tarjeta.innerHTML = `
  <h2>Producto</h2>
  <p>Descripción</p>
`;
```

Esta opción puede parecer muy atractiva porque reduce la cantidad de código visible.

11.2. El problema de fondo

Cuando utilizas `innerHTML`, el navegador interpreta esa cadena como HTML. Eso significa que:

- pierdes parte del control explícito sobre cómo se construye cada nodo;
- mezclas con más facilidad estructura y datos;
- y, si el contenido procede de fuentes no confiables, puedes introducir riesgos de seguridad.

11.3. Ejemplo problemático

```
const nombre = '';
const contenedor = document.querySelector("#contenedor");

contenedor.innerHTML = `<p>${nombre}</p>`;
```

En ese caso el navegador no trata ese contenido como texto literal, sino como HTML interpretable.

11.4. Alternativa más segura para texto

```
const nombre = '';
const contenedor = document.querySelector("#contenedor");

const parrafo = document.createElement("p");
parrafo.textContent = nombre;

contenedor.append(parrafo);
```

Aquí el contenido se trata como texto, no como marcado ejecutable.

11.5. Por qué en esta sesión priorizamos otras alternativas

En esta sesión se priorizan:

- `createElement()` ;
- `textContent` ;
- `setAttribute()` ;
- `append()` ;

porque permiten:

- construir el DOM paso a paso;
- controlar mejor cada nodo;
- separar mejor datos y renderizado;
- y adoptar una base más clara y segura para trabajar con interfaces dinámicas.

11.6. Comparativa rápida

Enfoque	Ventaja principal	Riesgo o limitación
<code>innerHTML</code>	rapidez al insertar estructuras	menos control y más riesgo si el contenido no es confiable
<code>createElement()</code> + <code>textContent</code> + <code>append()</code>	control, claridad y mayor seguridad	más código, pero mejor estructurado para aprender y mantener

Precaución importante

`innerHTML` no es “incorrecto” por definición, pero sí conviene usarlo con prudencia. Cuando el objetivo es construir interfaz dinámica a partir de datos, `createElement()` y `textContent` suelen ofrecer una base más controlable y pedagógicamente más sólida.

Referencias específicas del subtema

- MDN Web Docs — `Element.innerHTML`
 - MDN Web Docs — `Node.textContent`
 - MDN Web Docs — XSS (introducciones y referencias de seguridad relacionadas desde MDN)
-

12. Noción básica de reflow y repaint

12.1. Por qué aparece este concepto

Cada cambio visual o estructural en el DOM puede implicar trabajo adicional por parte del navegador. Dos ideas que suelen mencionarse son:

- **reflow**, relacionado con el recálculo del layout;
- **repaint**, relacionado con el repintado visual.

12.2. Qué debes entender en esta fase

No necesitas dominar ingeniería de rendimiento del navegador, pero sí una idea básica:

modificar muchas veces el DOM de forma desordenada puede costar más que agrupar cambios con criterio.

12.3. Consecuencia práctica

Por eso tiene sentido:

- evitar inserciones innecesarias;
- construir contenido de forma ordenada;
- usar herramientas como `DocumentFragment` cuando encajen;
- separar datos y renderizado.

Referencias específicas del subtema

- MDN Web Docs — Rendering performance
 - MDN Web Docs — DocumentFragment
-

13. Separar datos y renderizado

13.1. Una idea de diseño importante

Cuando una función mezcla:

- selección de nodos;
- construcción de elementos;
- lectura de datos;
- lógica de negocio;
- validaciones;

- cambios visuales;

el código se vuelve difícil de mantener.

13.2. Organización más clara

Conviene distinguir, al menos conceptualmente:

- los **datos**;
- la **función de render**;
- los **cambios de interfaz**.

13.3. Ejemplo sencillo

```
const productos = [
  { nombre: "Teclado", precio: 30 },
  { nombre: "Ratón", precio: 15 }
];

const lista = document.getElementById("lista-productos");

function renderizarProductos(productos) {
  lista.innerHTML = "";

  productos.forEach((producto) => {
    const tarjeta = document.createElement("article");
    const titulo = document.createElement("h2");
    const precio = document.createElement("p");

    titulo.textContent = producto.nombre;
    precio.textContent = `${producto.precio} €`;

    tarjeta.append(titulo, precio);
    lista.append(tarjeta);
  });
}

renderizarProductos(productos);
```

13.4. Por qué esto prepara React

Más adelante trabajarás con una idea muy parecida: la interfaz como representación del estado de los datos. Empezar a separar la construcción visual de la lógica de datos es una excelente preparación conceptual.

Idea clave

Cuando los datos cambian, el renderizado debería poder rehacerse con claridad. Esa relación entre datos e interfaz es una de las grandes ideas del desarrollo frontend moderno.

Referencias específicas del subtema

- MDN Web Docs — Introduction to the DOM
- MDN Web Docs — `Document.createElement()`

14. Ejemplo integrador de sesión

A continuación tienes un ejemplo que reúne varios conceptos trabajados.

```
const productos = [
  { nombre: "Teclado", precio: 30, destacado: true },
  { nombre: "Ratón", precio: 15, destacado: false },
  { nombre: "Monitor", precio: 200, destacado: true }
];

const lista = document.getElementById("lista-productos");

function crearTarjetaProducto(producto) {
  const tarjeta = document.createElement("article");
  const titulo = document.createElement("h2");
  const precio = document.createElement("p");

  tarjeta.classList.add("tarjeta-producto");

  if (producto.destacado) {
    tarjeta.classList.add("destacada");
  }

  titulo.textContent = producto.nombre;
  precio.textContent = `${producto.precio} €`;

  tarjeta.append(titulo, precio);

  return tarjeta;
}
```

```
function renderizarProductos(productos) {
  lista.innerHTML = "";

  const fragmento = document.createDocumentFragment();

  productos.forEach((producto) => {
    const tarjeta = crearTarjetaProducto(producto);
    fragmento.append(tarjeta);
  });

  lista.append(fragmento);
}

renderizarProductos(productos);
```

Qué se está aplicando aquí

Elemento	Uso en el ejemplo
Selección de nodo	acceso al contenedor con <code>getElementById()</code>
Creación de elementos	uso de <code>createElement()</code>
Clases dinámicas	<code>classList.add()</code> según datos
Contenido textual	<code>textContent</code>
Inserción agrupada	<code>DocumentFragment</code>
Renderizado dinámico	construcción visual a partir del array <code>productos</code>
Separación de responsabilidades	una función crea la tarjeta y otra renderiza el conjunto

15. Análisis técnico avanzado

15.1. Seleccionar bien simplifica todo lo demás

Una mala estrategia de selección genera código frágil. Por eso conviene:

- usar identificadores claros cuando tenga sentido;
- aprovechar selectores CSS bien pensados;
- no depender de estructuras demasiado ambiguas.

15.2. `textContent` y `innerHTML` no son equivalentes

Aunque ambas propiedades afectan al contenido, no resuelven el mismo problema:

- `textContent` trabaja con texto plano;
- `innerHTML` interpreta HTML.

Elegir mal entre ambas puede afectar a seguridad, claridad e intención.

15.3. Renderizar desde datos cambia la forma de pensar la interfaz

Una vez que comprendes que un array de objetos puede transformarse en una lista visual, dejas de pensar en HTML estático como único punto de partida. Empiezas a pensar en la interfaz como salida de una estructura de datos.

15.4. Rendimiento y claridad deben avanzar juntos

No tiene sentido obsesionarse con microoptimizaciones prematuras, pero tampoco ignorar que modificar el DOM repetidamente tiene coste. Lo razonable en esta fase es escribir código claro y empezar a adoptar pequeñas decisiones sensatas.

15.5. La estructura del código importa tanto como el resultado visual

Una interfaz puede “funcionar” y, aun así, estar mal organizada internamente. Separar responsabilidades desde el principio prepara mejor el camino hacia aplicaciones más complejas.

Referencias específicas del subtema

- MDN Web Docs — `Node.textContent`
- MDN Web Docs — `Element.innerHTML`
- MDN Web Docs — `DocumentFragment`
- MDN Web Docs — Rendering performance

16. Errores frecuentes al comenzar

Errores muy habituales

Estos fallos aparecen con frecuencia al empezar a manipular el DOM y construir interfaz dinámica.

16.1. Intentar modificar un elemento que no se ha seleccionado correctamente

```
const titulo = document.querySelector(".titulo-principal");
titulo.textContent = "Nuevo título";
```

Si el selector no encuentra nada, `titulo` será `null` y el código fallará al intentar acceder a `textContent`.

16.2. Confundir `querySelector()` con `querySelectorAll()`

`querySelector()` devuelve un único elemento.

`querySelectorAll()` devuelve una colección.

```
const productos = document.querySelectorAll(".producto");
console.log(productos.textContent); // Incorrecto conceptualmente
```

16.3. Usar `innerHTML` para todo

Aunque resulte cómodo, abusar de `innerHTML` puede empeorar la claridad y generar riesgos cuando el contenido procede de fuentes no controladas.

16.4. Mezclar datos, renderizado y lógica sin separación mínima

Cuando todo ocurre en un único bloque grande de código, el resultado se vuelve difícil de leer, de modificar y de depurar.

16.5. Reinsertar contenido sin limpiar el anterior cuando era necesario

Si vuelves a renderizar una colección sin vaciar el contenedor, puedes duplicar resultados involuntariamente.

16.6. Olvidar que algunos métodos modifican directamente el DOM

Cuando insertas, eliminas o alteras clases, estás cambiando la interfaz real del documento, no una copia abstracta. Conviene ser consciente de ese efecto inmediato.

17. Síntesis de la sesión

En esta sesión has trabajado el paso fundamental que conecta los datos con la interfaz:

- comprender el DOM como estructura programable del documento;
- seleccionar elementos con distintos mecanismos;
- leer y modificar contenido;
- trabajar con clases y atributos;
- crear, insertar y eliminar nodos;
- renderizar colecciones a partir de datos;
- vaciar y repintar contenido;
- introducir `DocumentFragment` como mejora práctica;
- distinguir entre `textContent` e `innerHTML`;
- comprender mejor por qué `createElement()` y `textContent` ofrecen una base más controlada y segura para construir interfaz;
- empezar a considerar cuestiones básicas de rendimiento y seguridad;
- separar datos y renderizado como criterio de diseño.

Esta sesión es especialmente importante porque muchas de las ideas que aparecerán después en eventos, formularios, asincronía y React se apoyan en la comprensión de cómo la interfaz puede generarse y actualizarse a partir del estado de los datos.

✓ **Al terminar esta sesión deberías ser capaz de...**

- explicar qué es el DOM y por qué JavaScript puede modificarlo;
- seleccionar nodos con `getElementById()`, `querySelector()` y `querySelectorAll()`;
- modificar texto, clases y atributos;
- crear nodos con `createElement()` e insertarlos en la página;
- eliminar nodos del documento;
- renderizar una lista de elementos a partir de un array de objetos;
- distinguir cuándo conviene usar `textContent` o `innerHTML`;
- comprender de forma inicial por qué conviene agrupar inserciones y separar datos del renderizado;
- reconocer el patrón general de renderizado que después aplicarás a datos externos y remotos.

18. Referencias documentales generales

Referencias oficiales y de consulta

- MDN Web Docs — Introduction to the DOM
- MDN Web Docs — Document Object Model (DOM)
- MDN Web Docs — `document`
- MDN Web Docs — `Document.getElementById()`
- MDN Web Docs — `Document.querySelector()`
- MDN Web Docs — `Document.querySelectorAll()`
- MDN Web Docs — `Element.closest()`
- MDN Web Docs — `Node.textContent`
- MDN Web Docs — `Element.innerHTML`
- MDN Web Docs — `Element.classList`
- MDN Web Docs — `Element.setAttribute()`
- MDN Web Docs — `Element.getAttribute()`
- MDN Web Docs — `Document.createElement()`
- MDN Web Docs — `Element.append()`
- MDN Web Docs — `Element.prepend()`
- MDN Web Docs — `Element.remove()`
- MDN Web Docs — `Document.createDocumentFragment()`
- MDN Web Docs — `DocumentFragment`

Recomendación de estudio

Al repasar esta sesión, no te limites a observar lo que aparece en pantalla. Intenta pensar siempre en los pasos internos: qué datos tienes, qué nodos seleccionas, qué elementos creas, dónde los insertas y qué parte del DOM cambia en cada momento. Comprender esa secuencia te ayudará muchísimo en las próximas sesiones y te preparará para entender mejor el enfoque declarativo de React.